DP Computer Science: A 4.3.8 Artificial Neural Networks (ANNs)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Describe an **Artificial Neural Network (ANN)** as a model inspired by the biological brain
- Outline the structure and function of a **single perceptron**, identifying its **inputs, weights, bias, activation function, and output**
- **Sketch a diagram** of a single perceptron
- Outline the structure of a **Multi-Layer Perceptron (MLP)** , identifying the **input, hidden, and output layers**
- Explain that **multi-layer networks** are used to model more complex, non-linear patterns than a single perceptron can

Key Vocabulary

Term	Definition
Artificial Neural Network (ANN)	A computational model inspired by the biological brain, composed of interconnected artificial neurons
Perceptron	The simplest form of a neural network; a single artificial neuron
Input	The data features fed into the perceptron
Weights	Parameters that determine the importance of each input
Bias	A parameter that adjusts the threshold for activation
Activation function	A function that determines whether the neuron "fires" (outputs a signal)
Output	The final prediction of the perceptron
Multi-Layer Perceptron (MLP)	A neural network with one or more hidden layers between input and output
Input layer	The first layer; receives the input data
Hidden layer	Intermediate layers where computation occurs
Output layer	The final layer; produces the prediction

Approaches to Learning (ATL) Elements

Skill	Description
Thinking (Abstraction & Conceptual Understanding)	Grasping the abstract model of a neuron and understanding how interconnected layers can represent complex functions
Communication	Using diagrams and analogies to explain the structure and information flow within a neural

Skill	Description
	network

---

2. Introduction: The Biological Inspiration

The Hook: Biological vs. Artificial Neurons

```text
┌──────────────────────────────────────────────────────────────────┐
│          BIOLOGICAL vs. ARTIFICIAL NEURONS                     │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐   │
│    │             BIOLOGICAL NEURON                         │  │
│    │                                                         │   │
│    │    ┌─────────────────────────────────────────────────┐  │   │
│    │    │  Dendrites (receive signals from other neurons) │  │   │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Cell Body (processes signals)                  │  │   │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Axon (transmits signal)                        │  │   │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Synapse (connects to other neurons)            │  │   │
│    │    └─────────────────────────────────────────────────┘  │   │
│    └─────────────────────────────────────────────────────────┘   │
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐   │
│    │             ARTIFICIAL NEURON                         │  │
│    │                                                         │   │
│    │    ┌─────────────────────────────────────────────────┐  │   │
│    │    │  Inputs (like dendrites)                        │  │   │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Weights + Bias (process importance)          │  │  │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Activation Function (decision to fire)       │  │  │
│    │    │          │                                      │  │   │
│    │    │          ▼                                      │  │   │
│    │    │  Output (transmits signal)                      │  │   │
│    │    └─────────────────────────────────────────────────┘  │   │
│    └─────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Key Insight

> *"Artificial Neural Networks are inspired by the brain—they use interconnected 'neurons' to process and learn from data."*

---

3. The Single Perceptron

What is a Perceptron?

> A perceptron is the simplest form of a neural network—a single artificial neuron that takes inputs, processes them, and produces an output.

Perceptron Structure

```text
┌─────────────────────────────────────────────────────────────┐
│              PERCEPTRON STRUCTURE                           │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│                 ┌───────────────────────────┐               │
│                 │         PERCEPTRON        │               │
│                 └───────────────────────────┘               │
│                                                             │
│   Input 1 ────►  ┌─────────────┐                            │
│                  │  Weight 1   │                            │
│   Input 2 ────►  ├─────────────┤                            │
│                  │  Weight 2   │                            │
│   Input 3 ────►  ├─────────────┤                            │
│                  │  Weight 3   │                            │
│   Input n ────►  ├─────────────┤                            │
│                  │  Weight n   │                            │
│                  └──────┬──────┘                            │
│                         │                                   │
│                  ┌──────┴──────┐                            │
│                  │   Sum       │                            │
│                  │  Σ (Input×  │                            │
│                  │   Weight)   │                            │
│                  │   + Bias    │                            │
│                  └──────┬──────┘                            │
│                         │                                   │
│                  ┌──────┴──────┐                            │
│                  │ Activation  │                            │
│                  │ Function    │                            │
│                  └──────┬──────┘                            │
│                         │                                   │
│                         ▼                                   │
│                  ┌─────────────┐                            │
│                  │   OUTPUT    │                            │
│                  └─────────────┘                            │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Components of a Perceptron

Component	Symbol	Description	Analogy
Inputs	$x_1, x_2, \ldots, x_n$	The data features being processed	What you see/hear
Weights	$w_1, w_2, \ldots, w_n$	Determine importance of each input	How much you trust each source
Bias	b	Adjusts the threshold for firing	Your natural tendency to say yes/no
Sum	$\sum(x_i \times w_i) + b$	Total evidence before decision	Adding up all the opinions
Activation Function	$f(x)$	Makes the final firing decision	The final decision-maker
Output	y	The final prediction	Your final decision

The Perceptron Calculation

> A perceptron computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function to produce an output.

```python
# Step 1: Weighted sum
z = (x1 * w1) + (x2 * w2) + ... + (xn * wn) + b

# Step 2: Activation function
output = activation_function(z)
```

Common Activation Functions

Activation Function	Description	Example
Step Function	Outputs 0 or 1 based on a threshold	If z > threshold, output 1; else 0
Sigmoid	Outputs between 0 and 1 (smooth curve)	S-shaped curve for probability
ReLU	Outputs 0 for negative values, z for positive	Max(0, z)
Tanh	Outputs between -1 and 1	Hyperbolic tangent

---

4. Interactive Activity: "The Perceptron Calculator"

Instructions

1. You are given a **fully-defined perceptron**:
   - 2 inputs: x_1 and x_2
   - Weights: $w_1 = 2$, $w_2 = -1$
   - Bias: $b = 1$
   - Activation function: **Step function** (output = 1 if z > 0; otherwise 0)
2. For each set of inputs, **calculate the output**:
   - Multiply inputs by weights
   - Add the bias
   - Apply the activation function

Example Calculation

Step	Calculation	Result
Input	$x_1 = 2, x_2 = 1$	
Weighted Sum	$(2 \times 2) + (1 \times -1) + 1$	$4 - 1 + 1 = 4$
Activation	$z = 4 > 0$	**Output = 1**

Practice Examples

Input Set	Weighted Sum	Output
$x_1 = 1, x_2 = 2$	$(1 \times 2) + (2 \times -1) + 1 = 2 - 2 + 1 = 1$	**1** (since 1 > 0)
$x_1 = 0, x_2 = 3$	$(0 \times 2) + (3 \times -1) + 1 = 0 - 3 + 1 = -2$	**0** (since -2 ≤ 0)

Input Set	Weighted Sum	Output
$x_1 = -1, x_2 = -1$	$(-1 \times 2) + (-1 \times -1) + 1 = -2 + 1 + 1 = 0$	**0** (since 0 is not > 0)

---

5. The Limitations of a Single Perceptron

Linear Separability

> A single perceptron can only solve problems that are linearly separable—problems where a straight line can separate the two classes.

```text
┌─────────────────────────────────────────────────────────────────┐
│              LINEAR SEPARABILITY                              │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              LINEARLY SEPARABLE                        │  │
│    │                                                         │  │
│    │    ● ● ● ● ●                                          │  │
│    │    ● ● ● ● ●                                          │  │
│    │    ──────┬───────                                      │  │
│    │    ○ ○ ○ ○ ○                                          │  │
│    │    ○ ○ ○ ○ ○                                          │  │
│    │                                                         │  │
│    │    One straight line can separate the classes          │  │
│    │    Perceptron can solve this!                         │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              NOT LINEARLY SEPARABLE                    │  │
│    │                                                         │  │
│    │    ● ● ○ ● ●                                          │  │
│    │    ● ● ○ ● ●                                          │  │
│    │    ○ ○ ● ○ ○                                          │  │
│    │    ● ● ○ ● ●                                          │  │
│    │                                                         │  │
│    │    No single line can separate the classes            │  │
│    │    A single perceptron CANNOT solve this!            │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

The XOR Problem

```text
┌─────────────────────────────────────────────────────────────────┐
│              THE XOR PROBLEM                                  │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                                                         │  │
│    │    XOR Truth Table:                                    │  │
│    │                                                         │  │
│    │    Input 1 │ Input 2 │ Output (XOR)                    │  │
│    │    ────────┼─────────┼──────────────                  │  │
│    │      0     │    0    │    0                            │  │
│    │      0     │    1    │    1                            │  │
│    │      1     │    0    │    1                            │  │
│    │      1     │    1    │    0                            │  │
```

```
│    └─────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                                                         │  │
│    │    ● ● ○ ● ●                                            │  │
│    │    ○ ○ ● ○ ○                                            │  │
│    │                                                         │  │
│    │    No single line can separate the classes!             │  │
│    │    A perceptron cannot learn XOR.                       │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

---

6. Multi-Layer Perceptrons (MLPs)

What is an MLP?

> A Multi-Layer Perceptron (MLP) is a neural network with one or more hidden layers between the input and output layers. Hidden layers allow the network to learn complex, non-linear patterns.

MLP Structure

```text
┌──────────────────────────────────────────────────────────────────┐
│              MULTI-LAYER PERCEPTRON STRUCTURE                    │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                                                         │  │
│    │    Input Layer    Hidden Layer    Output Layer          │  │
│    │                                                         │  │
│    │        ○──────────○──────────────○                      │  │
│    │        │          │              │                      │  │
│    │        ○──────────○──────────────○                      │  │
│    │        │          │              │                      │  │
│    │        ○──────────○──────────────○                      │  │
│    │        │          │              │                      │  │
│    │        ○──────────○──────────────○                      │  │
│    │                                                         │  │
│    │        Inputs      Processing     Outputs               │  │
│    │        (Features)  (Computation)  (Predictions)         │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

The Three Layers

Layer	Role	Description
Input Layer	Receives data	The number of neurons equals the number of input features
Hidden Layer(s)	Processes data	Extracts patterns; models complex relationships
Output Layer	Produces prediction	The number of neurons equals the number of outputs

Why Hidden Layers Matter

```text

┌─────────────────────────────────────────────────────────────────┐
│              WHY HIDDEN LAYERS MATTER                          │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌──────────────────────────────────────────────────────────┐ │
│    │  A single perceptron = linear decision boundary        │ │
│    │  Hidden layers = non-linear decision boundaries        │ │
│    │                                                          │ │
│    │  Without hidden layers:                                │ │
│    │                                                          │ │
│    │    ● ● ● ● ●                                           │ │
│    │    ○ ○ ○ ○ ○                                           │ │
│    │    Simple classification                               │ │
│    │                                                          │ │
│    │  With hidden layers:                                   │ │
│    │                                                          │ │
│    │    ● ● ○ ● ●                                           │ │
│    │    ● ● ○ ● ●                                           │ │
│    │    ○ ○ ● ○ ○                                           │ │
│    │    Complex patterns!                                   │ │
│    └──────────────────────────────────────────────────────────┘ │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Example: Solving XOR with an MLP

Layer	Neurons	Function
Input Layer	2	Takes the two input values
Hidden Layer	2	Learns to transform the input space
Output Layer	1	Produces the XOR output

---

7. Biological Neuron vs. Artificial Neuron

Comparison

```text

┌─────────────────────────────────────────────────────────────────┐
│         BIOLOGICAL vs. ARTIFICIAL NEURON COMPARISON           │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌──────────────────────────────────────────────────────────┐ │
│    │              BIOLOGICAL NEURON                         │ │
│    │                                                          │ │
│    │    Component         │   Function                      │ │
│    │    ──────────────────┼──────────────────────────────── │ │
│    │    Dendrites         │   Receives signals              │ │
│    │    Cell Body         │   Processes signals             │ │
│    │    Axon              │   Transmits signal              │ │
│    │    Synapse           │   Connects to other neurons     │ │
│    └──────────────────────────────────────────────────────────┘ │
│                                                                 │
│    ┌──────────────────────────────────────────────────────────┐ │
```

```
│    │            ARTIFICIAL NEURON                    │  │
│    │                                                  │  │
│    │    Component        │   Function                 │  │
│    │    ─────────────────┼──────────────────────────  │  │
│    │    Inputs           │   Receives data            │  │
│    │    Weights/Bias     │   Processes importance     │  │
│    │    Activation Func. │   Decides to "fire"        │  │
│    │    Output           │   Transmits result         │  │
│    └──────────────────────────────────────────────────┘  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

---

8. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Recap & Hook	10 min	Recap RL & GAs; show biological neuron vs. artificial neuron; ask for similarities
Lecture: Single Perceptron	15 min	Break down components: inputs, weights, bias, activation function, output
Interactive Activity	15 min	"Perceptron Calculator" — students calculate output for given perceptron
Lecture: MLPs	15 min	Explain limitations of single perceptron (XOR problem); introduce MLPs; input, hidden, output layers
Consolidation	5 min	Recap; preview next lesson

Discussion Questions

Question	Purpose
"What are the components of a perceptron?"	Identify components
"What does the activation function do?"	Understand its role
"Why can't a single perceptron solve XOR?"	Understand limitations
"What is the purpose of hidden layers?"	Understand MLPs
"How is an ANN inspired by the brain?"	Understand inspiration

---

9. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions

Assessment Type	Description
"Perceptron Calculator" Activity	Assess understanding of perceptron calculation
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What are the five components of a perceptron?
2. What is the purpose of the activation function?
3. Why is a single perceptron limited?
4. What is the role of hidden layers in an MLP?

---

10. Differentiation

Level	Strategy
Support	Use clear diagrams and analogies; step through the "Perceptron Calculator" activity carefully
Challenge	Research different activation functions; explore how weights are learned (backpropagation)

---

11. Resources

- **Presentation** (Slide Deck)
- **eBook** (A4.3.8 with examples)
- **Worksheet:** Perceptron calculations and MLP structure
- **Worksheet Answers**
- **Quizlet** (A4.3.8 vocabulary flashcards)

---

12. Summary: Key Takeaways

Concept	Key Point
ANN	Model inspired by the biological brain
Perceptron	Single artificial neuron
Components	Inputs, weights, bias, activation function, output
Weights	Importance of each input
Bias	Adjusts the threshold
Activation Function	Decision-maker (firing mechanism)
Single Perceptron	Can only solve linearly separable problems
MLP	Multi-layer network; can solve complex problems
Hidden Layers	Allow learning of non-linear patterns

```text
┌─────────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                               │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ✅ ANN = inspired by the brain                              │
│    ✅ Perceptron = single artificial neuron                    │
│    ✅ Inputs = data features                                   │
│    ✅ Weights = importance of each input                       │
│    ✅ Bias = threshold adjustment                              │
│    ✅ Activation Function = firing decision                    │
│    ✅ Single perceptron = linearly separable only              │
│    ✅ MLP = multiple layers (input, hidden, output)            │
│    ✅ Hidden layers = learn complex patterns                   │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

> *"Artificial Neural Networks are powerful models built from simple, interconnected units that can learn complex patterns by adjusting their weights and biases."*